

Guía de Arquitectura — Sistema de Gestión de Finanzas

Inversiones Compunet Segura, C.A. (Área de Papelería)

Cómo usar esta guía: léela completa una vez, y luego abre cada archivo mencionado mientras relees. La meta es que puedas explicar cualquier parte del sistema con tus propias palabras, sin memorizar.

1. ¿Qué es el sistema, en una frase?

Un sistema web de gestión de ingresos, egresos y facturación (punto de venta) para una papelería, con manejo de inventario y conversión automática de dólares a bolívares usando la tasa oficial del BCV.

2. Tecnologías usadas (el "stack")

Capa	Tecnología	¿Por qué?
Servidor / Lógica	PHP 8	Lenguaje sencillo de desplegar (funciona en cualquier hosting barato o XAMPP), ideal para sistemas pequeños y medianos.
Base de datos	MySQL	Relacional, gratuita, la más usada junto a PHP. Soporta transacciones (importante para facturación).
Interfaz	HTML + CSS + JavaScript puro	Sin frameworks: menos dependencias, más control y todo el código es entendible.
Gráficos	Chart.js	Librería ligera para los gráficos de estadísticas.
Acceso a datos	PDO con prepared statements	Previene inyección SQL y separa datos de consultas.

Patrón general: el sistema separa **vistas** (páginas PHP que muestran HTML) de **endpoints** (archivos PHP que devuelven JSON). El JavaScript de cada página llama a los endpoints con `fetch()` y pinta los resultados. Es el mismo concepto de una API REST, simplificado.

3. ¿Cómo viaja una petición? (flujo típico)

Ejemplo: el usuario abre "Ver Ingresos".

Navegador

| 1. GET /listar_ingresos.php



listar_ingresos.php — incluye → includes/auth.php (¿hay sesión? si no → login)
— incluye → includes/header.php (menú + widget tasa BCV)
— incluye → includes/footer.php (modales + JS común)

| 2. El navegador ejecuta listar-movimientos.js



| 3. fetch("obtener_movimientos.php?tipo=ingreso&desde=...&hasta=...")



obtener_movimientos.php — incluye → conexion.php (conecta BD + auto-instalador)
— incluye → verificar_sesion_api.php (protege el endpoint)
— consulta MySQL con PDO y devuelve JSON

| 4. El JS recibe el JSON y arma la tabla fila por fila



Usuario ve la tabla

Este mismo patrón se repite en TODO el sistema. Si entiendes este flujo, entiendes el 80% de la arquitectura.

4. Módulos, uno por uno

4.1 Autenticación (login)

- **Archivos:** login.php , procesar_login.php , cerrar_sesion.php , includes/auth.php , includes/verificar_sesion_api.php
- Las contraseñas **nunca se guardan en texto plano**: se usa password_hash() con **BCRYPT** (algoritmo de hash lento a propósito, resistente a fuerza bruta) y se verifican con password_verify() .
- Tras el login se guarda el usuario en \$_SESSION . Toda página incluye auth.php (redirige al login si no hay sesión) y todo endpoint incluye verificar_sesion_api.php (devuelve error JSON si no hay sesión).

4.2 Layout compartido (la "plantilla")

- **Archivos:** includes/header.php , includes/footer.php , assets/css/estilos.css
- Cada página define 2-3 variables (\$paginaActiva , \$tituloPagina , \$incluirChartJs) y luego incluye el header. Así el menú de navegación, los estilos y el widget de la tasa BCV viven en UN solo lugar.
- El footer incluye los modales compartidos (agregar concepto, banco, cliente, proveedor, tasa) y los scripts comunes.

4.3 Servicio de Tasa BCV 🌟 (pregunta segura del jurado)

- **Archivos:** includes/tasa_bcv.php , obtener_tasa_bcv.php , guardar_tasa_bcv.php
- **Lógica de prioridad** (función obtenerTasaBCV):

1. Si hoy hay una tasa fijada **manualmente** → se usa esa (el administrador manda).
 2. Si no, se busca la tasa de hoy en la tabla `tasas_bcv` (caché: la API se consulta UNA vez al día, no en cada operación).
 3. Si no hay tasa de hoy, se consulta la **API pública** (`ve.dolarapi.com`) con timeout de 4 segundos y se guarda en la tabla.
 4. Si la API falla (sin internet), se usa la **última tasa conocida**.
- **Punto clave:** cada movimiento y factura guarda la tasa con la que se hizo (`tasa_bcv` , `monto_bs`). Así el histórico en bolívares es correcto aunque la tasa cambie mañana. El cálculo se hace **en el servidor**, nunca se confía en el valor que mande el navegador.
 - En el frontend, `common.js` publica la tasa en `window.TASA_BCV` y dispara el evento `tasa-bcv-lista` para que cualquier página se actualice (esto es la "gestión de estado global" del requerimiento).

4.4 Catálogo de Productos/Servicios

- **Archivos:** `productos.php` , `assets/js/productos.js` , `obtener_conceptos.php` , `insertar_concepto.php` , `actualizar_concepto.php` , `eliminar_concepto.php`
- La tabla `conceptos` guarda ambos tipos: **productos** (con precio y stock) y **servicios** (con precio, sin stock — `stock = NULL` significa "no se controla").
- CRUD completo con validación de nombres duplicados. No se puede eliminar un concepto que ya tenga movimientos o facturas (lo protege una llave foránea RESTRICT, y el sistema muestra un mensaje amigable).

4.5 Registrar Egreso (compras y gastos)

- **Archivos:** `registrar_egreso.php` , `assets/js/registrar-egreso.js` , `insertar_movimiento.php`
- **Nota de arquitectura:** antes existía un módulo gemelo "Registrar Ingreso". Se **eliminó** porque duplicaba la función del POS: hoy **todo ingreso entra únicamente por Facturación** (un solo punto de escritura contable y de inventario). `insertar_movimiento.php` **rechaza** cualquier intento de registrar un ingreso y solo acepta egresos.
- Al seleccionar un concepto se **autocompleta su precio** de catálogo (el dato viaja en el atributo `data-precio` de cada opción del select).
- El total se calcula reactivamente en USD **y en bolívares** al escribir.
- El servidor revalida todo (cantidad > 0, precio > 0, concepto existe) antes de insertar. **Regla de oro: nunca confiar en la validación del navegador.**

4.6 Módulo POS (Facturación múltiple) ★★ (el corazón del proyecto)

- **Archivos:** `facturacion.php` , `assets/js/facturacion.js` , `guardar_factura.php` , `obtener_correlativo.php`
- Es el **único punto de entrada de ingresos** del sistema (ver 4.5): toda venta, simple o múltiple, se registra aquí como una factura.
- **Cabecera consolidada:** cliente (o "Consumidor final", opcional), **referencia** libre (opcional, ej: número de transferencia) y correlativo de factura que se genera automático (`00001` , `00002` . . .) — al agregar el **primer ítem** al carrito el número se refresca desde el servidor, salvo que el usuario haya escrito uno a mano; el sistema rechaza números repetidos.

- **Buscador predictivo:** el catálogo se carga una vez y se filtra en memoria mientras se escribe, mostrando precio y stock de cada coincidencia.
- **Carrito reactivo:** cantidades y precios editables en línea; subtotales en \$ y Bs se recalculan al instante.
- **Pagos mixtos:** una factura puede pagarse con varias líneas (ej: mitad en efectivo USD y mitad en Pago Móvil en Bs). El sistema convierte con la tasa del día, muestra el "restante" y **no permite guardar si los pagos no cuadran** con el total (tolerancia de \$0.10 por redondeos).
- **Inventario:** al vender un producto se descuenta el stock. Si va a quedar negativo, el sistema **avisa pero permite la venta** (decisión de negocio: el inventario físico no siempre está al día y no se debe frenar una venta).
- Al guardar, además de la factura se generan **movimientos de ingreso** (uno por ítem) ligados a la factura, para que las ventas POS aparezcan en los listados, el resumen y las estadísticas automáticamente.

4.7 Ticket de impresión (80mm)

- **Archivo:** `factura_ticket.php`
- Página HTML con CSS para impresora térmica: `@page { size: 80mm auto }`, fuente monoespaciada, separadores punteados. Al abrirse lanza `window.print()` automáticamente. Desde el diálogo de impresión también se puede "Guardar como PDF".

4.8 Consultas y filtros (Ver Ingresos / Ver Egresos)

- **Archivos:** `listar_ingresos.php`, `listar_egresos.php`, `assets/js/listar-movimientos.js`, `obtener_movimientos.php`
- Filtros rápidos: **Hoy / Esta Semana / Este Mes / Este Año / Ver Todo** (calculan el rango de fechas en el navegador) + rango de fechas libre + **buscador de texto** que busca en el servidor (concepto, cliente, proveedor, fuente, número de factura y referencia) con `LIKE` parametrizado.
- La búsqueda usa *debounce* (espera 350ms tras dejar de escribir) para no bombardear al servidor con cada tecla.

4.9 Resumen y Estadísticas

- **Archivos:** `index.php`, `assets/js/resumen.js`, `estadisticas.php`, `assets/js/estadisticas.js`, `obtener_estadisticas.php`
- **Resumen:** tarjetas de Ingresos, Egresos y Balance (en \$ y su equivalente en Bs a la tasa del día) + tarjeta con la tasa BCV + historial completo.
- **Estadísticas:** gráfico de torta (proporción ingreso/egreso), barras por concepto, y **evolución mensual de los últimos 12 meses** con línea de balance.

4.10 Auto-instalador de base de datos ★ (diferenciador técnico)

- **Archivos:** `conexion.php`, `instalador.php`
- En cada conexión, el sistema verifica que existan todas las tablas, columnas y llaves foráneas, y **crea/repara lo que falte** consultando `INFORMATION_SCHEMA`. Esto significa que el sistema se instala solo en una máquina

nueva y que la base de datos nunca queda desactualizada respecto al código. (Es el equivalente artesanal de las "migraciones" de los frameworks grandes.)

5. Mapa de archivos

```
Felix_sistema/
├── index.php                ← Resumen (página principal)
├── facturacion.php          ← POS / Facturación (única entrada de ingresos)
├── registrar_egreso.php     ← Registro de egresos (compras/gastos)
├── listar_ingresos.php      ← Consulta con filtros
├── listar_egresos.php       ← Consulta con filtros
├── productos.php           ← CRUD del catálogo
├── estadisticas.php         ← Gráficos
├── factura_ticket.php       ← Ticket imprimible 80mm
├── login.php / procesar_login.php / cerrar_sesion.php
├──
├── conexion.php            ← Conexión BD + dispara auto-instalador
├── instalador.php          ← Crea/repara tablas, columnas y FKs
├──
├── obtener_*.php           ← Endpoints JSON de lectura
├── insertar_*.php          ← Endpoints JSON de escritura
├── actualizar_concepto.php / eliminar_*.php
├── guardar_factura.php      ← Guardado transaccional del POS
├── guardar_tasa_bcv.php / obtener_tasa_bcv.php / obtener_correlativo.php
├──
├── includes/
│   ├── auth.php            ← Protege páginas
│   ├── verificar_sesion_api.php ← Protege endpoints
│   ├── header.php / footer.php ← Layout compartido
│   └── tasa_bcv.php        ← Servicio de tasa (lógica central)
├──
└── assets/
    ├── css/estilos.css
    └── js/
        ├── common.js       ← Utilidades + estado global (tasa BCV)
        ├── filtro-mes.js    ← Helpers de fechas
        ├── facturacion.js   ← Lógica del POS
        ├── productos.js     ← Lógica del catálogo
        ├── registrar-egreso.js
        ├── listar-movimientos.js
        ├── resumen.js
        └── estadisticas.js
```